MODULE 23 · DOMAIN 3 - DEPLOYMENT

CodeDeploy and Deployment Strategies

Three compute platforms, three different appspec files, and lifecycle hooks that are not the same on any of them.

DOMAIN 3 of 4 EXAM WEIGHT 24% MODULE 23 of 34

admin@sthithapragna.com

CodeDeploy's job is **controlling exposure**. It decides how fast new code meets real traffic, and what happens when it goes wrong.

The three compute platforms

PLATFORM	HOW THE DEPLOYMENT WORKS
EC2/On-Premises	The agent runs scripts on each instance. In-place or blue/green
AWS Lambda	Traffic shifts between two versions behind an alias. Always blue/green in effect
Amazon ECS	A replacement task set is created and traffic shifts to it. Blue/green only

ON THE EXAM

Only EC2 supports in-place deployment. Lambda and ECS always create something new and shift traffic to it, which is why rollback on those is immediate.

Applications and deployment groups

- An **application** names the compute platform and contains deployment groups.
- A **deployment group** is a set of targets plus the settings used to deploy to them.
- For EC2, targets are chosen by **tags** or by Auto Scaling group.
- Each group has its own deployment configuration, alarms and rollback settings.
- So dev, staging and production are typically three groups in one application.

ON THE EXAM

Tag-based targeting means an instance joins a deployment group by carrying the right tag. A mistagged instance silently receives deployments it should not.

The appspec file

- **appspec.yml** for EC2, at the root of the application bundle.
- **appspec.yml or appspec.json** for Lambda and ECS, supplied to the deployment.
- **The structure is not the same across platforms.** An EC2 appspec is invalid for Lambda.
- EC2 has **files** and **permissions** sections. Lambda and ECS have a **Resources** section instead.
- All three have a **hooks** section, with different hooks available.

> ON THE EXAM
>
> This is the detail people miss. It is one filename and three different formats, and the exam shows you a fragment and asks which platform it is for.

The EC2 appspec sections

- **version** is 0.0, and **os** is linux or windows.
- **files** maps a source path in the bundle to a destination on the instance.
- **permissions** sets owner, group and mode on the copied files.
- **hooks** maps lifecycle events to scripts, each with an optional timeout and runas.
- A hook script that exceeds its timeout fails the deployment. The default is 3600 seconds.

> ON THE EXAM
>
> The files section is why an EC2 bundle needs a defined structure. CodeDeploy copies exactly what you list, to exactly where you say.

In-place deployment on EC2

- The application is stopped on each instance, updated, and started again.
- Instances are deregistered from the load balancer while being updated.
- **Capacity drops during the deployment**, by however much the configuration allows.
- **Rollback means deploying the previous version again.** It is not instant.
- Cheap, because no additional instances are launched.

> ON THE EXAM
>
> The same trade-off as Beanstalk's rolling policy in module 14: no extra cost, reduced capacity, and rollback is another full deployment.

EC2 deployment configurations

CONFIGURATION	BEHAVIOUR
`CodeDeployDefault.AllAtOnce`	Every instance at once. Fastest, and there is downtime
`CodeDeployDefault.HalfAtATime`	Half the fleet, then the other half. 50% capacity maintained
`CodeDeployDefault.OneAtATime`	One instance at a time. Slowest and safest
`Custom`	You define minimum healthy hosts, as a count or a percentage

ON THE EXAM

Minimum healthy hosts is the underlying setting. The named configurations are just presets over it, and a custom one lets you pick any point on the scale.

Blue/green on EC2

- New instances are launched, the application is deployed to them, then traffic is switched.
- The load balancer is repointed at the replacement instances all at once.
- **Rollback is repointing back**, so it takes seconds rather than a full deployment.
- Original instances are terminated after a configurable **wait time**, or kept.
- It briefly doubles your instance count, and therefore your cost.

> ON THE EXAM
>
> The termination wait time is the useful control: keep the old fleet for an hour, and rollback stays instant for that whole window.

EC2 lifecycle hooks, in order

HOOK	WHEN IT RUNS
ApplicationStop	Before anything. Runs the PREVIOUS deployment's script
DownloadBundle	CodeDeploy copies the bundle. You cannot script this
BeforeInstall	Before files are copied. Backups, decryption
Install	CodeDeploy copies the files. You cannot script this
AfterInstall	After files are copied. Configuration, permissions
ApplicationStart	Start the application
ValidateService	Verify the deployment worked. The last chance to fail it

ON THE EXAM

ApplicationStop is the one that surprises people: it runs the script from the previously deployed revision, because the new one is not on the instance yet.

The blue/green traffic hooks

- **BeforeBlockTraffic**, **BlockTraffic**, **AfterBlockTraffic** on the original instances.
- **BeforeAllowTraffic**, **AllowTraffic**, **AfterAllowTraffic** on the replacement instances.
- **BlockTraffic and AllowTraffic cannot be scripted.** CodeDeploy performs them.
- So four of the six are yours: the Before and After pairs.
- Use BeforeAllowTraffic to warm a cache, and AfterAllowTraffic to verify live traffic.

ON THE EXAM

The pattern across all platforms: the verbs themselves are CodeDeploy's, and the Before and After variants are yours. Install, DownloadBundle, BlockTraffic and AllowTraffic are never scriptable.

The CodeDeploy agent

- **EC2 and on-premises deployments require the agent.** Lambda and ECS do not.
- It polls CodeDeploy for work, downloads the bundle and runs the appspec hooks.
- Installable through user data, a custom AMI, or Systems Manager State Manager.
- If the agent is not running, the deployment simply **times out** rather than failing fast.
- The instance profile needs permission to read the artifact bucket.

> ON THE EXAM
>
> 'The deployment hung and eventually timed out' on EC2 is nearly always the agent: not installed, not running, or lacking permission to fetch the bundle.

Auto Scaling and deployments

- When a deployment group targets an Auto Scaling group, **new instances deploy automatically**.
- A scale-out event launches an instance, and CodeDeploy deploys the latest revision to it.
- That means an instance launched mid-deployment can receive the **wrong** revision.
- Scaling during a deployment is best suspended, which CodeDeploy can do for you.
- A failed deployment to a new instance causes it to be terminated and relaunched.

> ON THE EXAM
>
> The mid-deployment race is the subtle part: two sources of truth about what should be running, briefly disagreeing with each other.

Lambda deployments

- CodeDeploy shifts traffic between two **versions** of a function, using an **alias**.
- The alias is the stable ARN clients call. Weights on it determine the split.
- There are no instances, no agent and no files to copy.
- The appspec names the function, the alias, and the target and current versions.
- This is exactly what SAM's **DeploymentPreference** configures for you.

> ON THE EXAM
>
> Module 21 wrote four lines of YAML; this is what those four lines create. Module 24 covers the version and alias mechanics underneath.

Lambda deployment configurations

CONFIGURATION	BEHAVIOUR
`Canary10Percent5Minutes`	10% for 5 minutes, then the remaining 90%
`Canary10Percent30Minutes`	10% for 30 minutes, then the rest. Also 10 and 15 minute variants
`Linear10PercentEvery1Minute`	10% more every minute, over 10 minutes
`Linear10PercentEvery10Minutes`	10% more every 10 minutes, over 100 minutes
`AllAtOnce`	All traffic immediately. No gradual shift

ON THE EXAM

Canary shifts in two steps: a small slice, a pause, then everything. Linear shifts in equal increments throughout. The names state the numbers exactly.

Lambda hooks

- **BeforeAllowTraffic** runs before any traffic reaches the new version.
- **AfterAllowTraffic** runs after the shift has completed.
- That is the complete list. There is no install phase and no ValidateService.
- Each hook is a **Lambda function** you write, not a shell script.
- It must call **PutLifecycleEventHookExecutionStatus** with Succeeded or Failed.

> ON THE EXAM
>
> That last line matters: a hook that never reports its status leaves the deployment waiting. Returning normally from the function is not enough.

What a hook function receives

- The hook function receives a **DeploymentId** and a **LifecycleEventHookExecutionId**.
- It does **not** receive the new function version. You pass that in through environment variables.
- It must call **PutLifecycleEventHookExecutionStatus** with that execution ID.
- Its execution role needs **codedeploy:PutLifecycleEventHookExecutionStatus**.
- Returning Failed rolls the deployment back. Timing out does too, eventually.

> ON THE EXAM
>
> Missing that IAM permission is the classic first failure: the test passes, the status call is denied, and the deployment stalls until it times out.

ECS deployments

- CodeDeploy creates a **replacement task set** running the new task definition.
- The load balancer needs **two target groups**: one live, one for the replacement.
- An optional **test listener** on a separate port reaches the new tasks before real traffic does.
- That is what makes running integration tests against the new version possible.
- Traffic shifts by canary, linear or all at once, exactly as for Lambda.

> ON THE EXAM
>
> The test listener is the distinctive ECS capability. You can exercise the replacement task set with real requests on a separate port while production traffic is untouched.

Lifecycle hooks by platform

PLATFORM	HOOKS AVAILABLE TO YOU
EC2, in-place	ApplicationStop, BeforeInstall, AfterInstall, ApplicationStart, ValidateService
EC2, blue/green	The above, plus the Before and After variants of BlockTraffic and AllowTraffic
AWS Lambda	BeforeAllowTraffic, AfterAllowTraffic
Amazon ECS	BeforeInstall, AfterInstall, AfterAllowTestTraffic, BeforeAllowTraffic, AfterAllowTraffic

READ THIS TWICE

Worth screenshotting and memorising. AfterAllowTestTraffic exists only on ECS, and ValidateService exists only on EC2. Those two facts alone answer several questions.

Automatic rollback

- Roll back **on deployment failure**, or when a **CloudWatch alarm** enters ALARM state.
- A rollback is a **new deployment** of the previous revision, with its own deployment ID.
- For blue/green and Lambda that is near-instant, because the old version still exists.
- For in-place it is a full redeployment, and takes as long as one.
- A failing lifecycle hook triggers rollback too, which is why validation hooks matter.

> ON THE EXAM
>
> Alarm-based rollback is what makes a canary meaningful. Without an alarm, shifting 10% of traffic just means 10% of users see the failure for five minutes.

Deployment history and redeploying

- Every deployment keeps its **revision**, so a previous one can be redeployed directly.
- That is the manual equivalent of a rollback, and it is how you recover after the fact.
- **CreateDeployment** can target a specific revision rather than the latest.
- Deployment history shows every attempt, its configuration and which instances failed.
- A rolled-back deployment appears as **two** entries: the failure and the rollback.

ON THE EXAM

Worth knowing for troubleshooting questions: the history is the audit trail, and the rollback is a separate deployment with its own ID and its own outcome.

Alarms, triggers and stopping

- **Alarms** on a deployment group stop and roll back a deployment automatically.
- **Triggers** publish deployment and instance events to an **SNS topic**.
- CodeDeploy also emits **EventBridge events**, for anything more elaborate.
- **StopDeployment** halts a running deployment, optionally rolling back.
- A deployment can be configured to fail if the alarm state cannot be determined.

> ON THE EXAM
>
> Choose alarms that reflect user experience: error rate and latency. An alarm on CPU will not notice that every request is returning a 500.

Currency note: ECS built-in blue/green

- **The examinable answer:** blue/green on ECS uses **CodeDeploy**.
- Amazon ECS now offers **native blue/green deployments** without CodeDeploy.
- It has its own lifecycle hooks, invoked as Lambda functions, covering more stages.
- Hooks return SUCCEEDED, IN_PROGRESS or FAILED, and a FAILED result rolls back.
- **Answer CodeDeploy on the exam.** Exam guide v2.1 predates this entirely.

> ON THE EXAM
>
> The CodeDeploy path still works and is still supported. This is an additional option, not a replacement, but new ECS work increasingly uses the native route.

Every deployment mechanism in this course

WHERE	MECHANISM
Lambda function versions	CodeDeploy canary or linear, via an alias
ECS service	CodeDeploy blue/green with two target groups
EC2 fleet	CodeDeploy in-place or blue/green
Elastic Beanstalk	Deployment policies: rolling, immutable, traffic splitting
API Gateway stage	Canary release deployment on the stage
CloudFormation stack	Change sets, and rollback triggers on alarms
AppConfig	Deployment strategy with bake time and alarm rollback

ON THE EXAM

Seven mechanisms across seven services, and they are not interchangeable. Each operates on a different layer, which is exactly what questions test.

Choosing a strategy

- **Rollback must be immediate** means blue/green or traffic shifting. In-place cannot do it.
- **Validate before real users** means a test listener, or a BeforeAllowTraffic hook.
- **Cannot afford double capacity** rules out blue/green and immutable.
- **A percentage of traffic first** means canary or linear.
- **No downtime and no extra cost** is the one combination nothing gives you.

> ON THE EXAM
>
> That last line is worth stating plainly. Zero downtime is bought either with extra capacity or with reduced capacity. There is no third option.

Diagnosing a failed deployment

- The console shows **which lifecycle event failed** on which instance. Start there.
- On EC2, script output goes to the agent log on the instance.
- **A hang that times out** is nearly always the agent not running, or a hook not reporting status.
- **Failure at ApplicationStop** on a first deployment means the previous revision's script is broken or missing.
- A failing script must exit non-zero. A script that fails silently lets the deployment continue.

> ON THE EXAM
>
> The ApplicationStop case is a genuine trap: the failure is in code from the previous deployment, not the one you are trying to deploy.

Ask first: **which platform**? The hooks, the appspec and the rollback speed all change with it.

A Lambda function is deployed with a Canary10Percent10Minutes configuration. The team wants an automated smoke test to run against the new version and abort the deployment before any production traffic reaches it.

The smoke test calls the new version directly and checks several responses.

How should the smoke test be wired in?

A. A ValidateService hook in the appspec, running the smoke test script

B. A BeforeAllowTraffic hook, as a Lambda function reporting Succeeded or Failed

C. An AfterAllowTestTraffic hook, invoked once the test listener serves the new version

D. A CloudWatch alarm on the function's error metric, with automatic rollback enabled

◆ PAUSE AND DECIDE

How should the smoke test be wired in?

A ValidateService hook

ValidateService exists only on the EC2 platform. It is not available for a Lambda deployment at all.

B BeforeAllowTraffic hook

One of Lambda's two hooks, and it runs before any traffic shifts. Reporting Failed aborts the deployment before a single user is affected.

C AfterAllowTestTraffic hook

An ECS-only hook, tied to the test listener. Lambda deployments have no test listener and no such hook.

D Alarm with rollback

A useful safety net, and it reacts to errors after traffic has already reached the new version. The requirement is to test beforehand.

THE SIGNAL

Two distractors are hooks from other platforms. Identifying the platform first eliminates both immediately, which is why that habit is worth building.

> A new EC2 deployment group has been created. The first deployment to it fails almost immediately, with the console reporting that the ApplicationStop lifecycle event failed on every instance.
>
> The instances were previously used for a different application deployed by CodeDeploy some months ago. The new revision's scripts have been tested and work correctly.

What is the cause?

A. The CodeDeploy agent is not installed on the instances

B. The new revision's appspec.yml has invalid YAML

C. The instance profile lacks permission to read the artifact bucket

D. ApplicationStop runs the previous revision's script, which is broken or missing

◆ PAUSE AND DECIDE

What is the cause?

A Agent not installed

Without the agent the deployment would hang and time out, not report a specific lifecycle event failure. Reporting ApplicationStop proves the agent is running.

B Invalid appspec YAML

That fails before any lifecycle event runs. It also contradicts the statement that the new scripts work.

C Bucket permissions

That fails at DownloadBundle, which comes after ApplicationStop. The failure would name a different event.

D Previous revision's script

ApplicationStop runs from the last successfully deployed revision, because the new bundle is not on the instance yet. Old, stale scripts are the cause.

THE SIGNAL

The clue is 'previously used for a different application'. Those instances still hold that application's CodeDeploy state, and ApplicationStop runs from it.

> An ECS service must be deployed such that a full integration test suite runs against the new tasks, using real HTTP requests, before any production traffic reaches them. The tests take about four minutes.
>
> Production traffic must be entirely unaffected while the tests run, and a failing test must abort the deployment.

How should this be configured?

A. Run the tests in a CodeBuild stage before the deploy stage

B. Use a Linear10PercentEvery1Minute configuration so failures affect few users

C. Configure a test listener on a second port, with an AfterAllowTestTraffic hook running the tests

D. Add a ValidateService hook that runs the integration suite

◆ PAUSE AND DECIDE

How should this be configured?

A. CodeBuild stage before deploy

Tests a build artifact, not the deployed service. It cannot make real HTTP requests to tasks that do not exist yet.

B. Linear traffic shifting

Limits the blast radius rather than preventing it. Production traffic reaches the new tasks immediately, which the scenario forbids.

C. Test listener plus AfterAllowTestTraffic

The test listener serves the replacement task set on a separate port, so real requests reach it while production traffic does not. A failed hook aborts the deployment.

D. ValidateService hook

An EC2-only hook. It does not exist for ECS deployments.

THE SIGNAL

'Real HTTP requests' plus 'production traffic entirely unaffected' describes the test listener precisely. It is the one ECS capability nothing else replicates.

A trading application runs on a fleet of 20 EC2 instances. Deployments must cause no reduction in capacity, and a bad release must be reversible within seconds. The business will fund additional instances during a deployment.

The application cannot tolerate running two versions simultaneously against the same database.

Which deployment approach fits?

A. CodeDeploy blue/green, with the load balancer switched all at once and a termination wait time

B. CodeDeploy in-place with OneAtATime

C. CodeDeploy in-place with a custom configuration requiring 100% minimum healthy hosts

D. CodeDeploy blue/green with Canary10Percent5Minutes traffic shifting

◆ PAUSE AND DECIDE

Which deployment approach fits?

A Blue/green, switch all at once

Full capacity throughout, no period where both versions serve traffic, and rollback is repointing the load balancer at instances still running.

B In-place OneAtATime

Reduces capacity by one instance at a time, and rollback is a full redeployment taking twenty instance updates. Both requirements fail.

C In-place at 100% healthy

Requiring every instance to stay healthy means no instance can be taken out to update. The deployment cannot proceed at all.

D Blue/green with canary

The closest wrong answer. Canary shifting means both versions serve traffic during the shift, which the last sentence explicitly forbids.

THE SIGNAL

The final sentence is doing the work. It eliminates every gradual strategy, including the canary that would otherwise be the safest choice.

What to carry forward

- **Hooks differ by platform.** ValidateService is EC2 only. AfterAllowTestTraffic is ECS only.
- **ApplicationStop runs the previous revision's script**, which is why it fails on stale instances.
- **Install, DownloadBundle, BlockTraffic and AllowTraffic cannot be scripted.**
- Lambda and ECS roll back instantly. In-place EC2 rollback is a full redeployment.
- Zero downtime costs either extra capacity or reduced capacity. There is no third option.

> ON THE EXAM
>
> Next: Lambda versions, aliases and safe releases - the mechanics underneath the traffic shifting this module has been configuring.